Redis过期淘汰策略源码分析

课程目标

掌握Redis的过期策略以及原理

掌握Redis的淘汰策略以及原理

掌握Redis持久化机制以及数据恢复

总之，我们今天的课程学的就是基于Redis是内存型数据库，面试的时候必问的知识！

为什么要有过期策略跟淘汰策略

因为我们的redis是一个内存型数据库，我们的数据都是放在内存里面的！但是内存是有大小的！

比如，redis,昨天我们讲了有个很重要的配置文件，那么redis.conf里面有个配置，

```
# maxmemory <bytes>    redis占用的最大内存
```

如果我们不淘汰，那么它的数据就会满，满了肯定就不能再放数据，发挥不了redis的作用！

比如冰箱，你如果放满了，那么你的菜就不能放冰箱了！

那么今天我们会讲过期策略与淘汰策略，那么过期策略与淘汰策略的区别又是什么呢?

过期策略：拿出redis中已经过期了的数据，就像你从冰箱把坏的菜拿出来！！但是有一种情况，就是冰箱里面的菜都没坏，redis里面的数据都没过期，它也是会放满的，那怎么办?

那么当redis里面的数据都没过期。但是内存满了的时候，我们就得从未过期的数据里面去拿出一些扔掉，那么这个就是我们的淘汰策略！！！

我们接下来一个一个的讲！！

咕泡出品

Redis过期策略

惰性过期（被动过期）

这个怎么实现的呢？所谓惰性，是不是就很懒的意思，就是只有访问我的时候，我才会去判断过不过期，不然我懒得去判断，我不会主动去判断过没过期！

访问一个key时判断该 key 是否已过期，过期则清除。

该策略就可以最大化地节省CPU资源，因为它平时都懒得去判断，所以也没有啥cpu损耗，因为只有访问的时候我才去判断一下！

但是却对内存非常不友好。因为你不实时过期了，该过期删除的就可能一直堆积在内存里面！极端情况可能出现大量的过期key没有再次被访问，从而不会被清除，占用大量内存。

源码（expireIfNeeded db.c文件下1302行）

```
int expireIfNeeded(redisDb *db, robj *key) {
    if (!keyIsExpired(db,key)) return 0;

    /* If we are running in the context of a slave, instead of
     * evicting the expired key from the database, we return ASAP:
     * the slave key expiration is controlled by the master that will
     * send us synthesized DEL operations for expired keys.
     *
     * Still we try to return the right information to the caller,
     * that is, 0 if we think the key should be still valid, 1 if
     * we think the key is expired at this time. */
    if (server.masterhost != NULL) return 1;

    /* Delete the key */
    server.stat_expiredkeys++;
    propagateExpire(db,key,server.lazyfree_lazy_expire);
```

```
    notifyKeyspaceEvent(NOTIFY_EXPIRED,
        "expired",key,db->id);
    int retval = server.lazyfree_lazy_expire ?
dbAsyncDelete(db,key) :

dbSyncDelete(db,key);
    if (retval) signalModifiedKey(NULL,db,key);
    return retval;
}
```

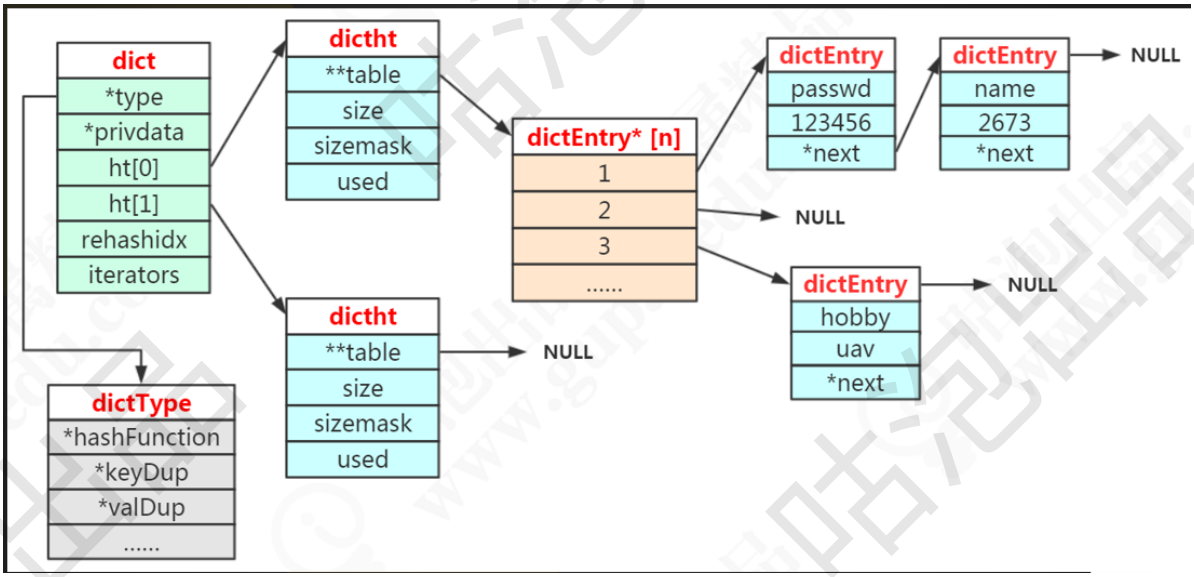
我们刚才说了主动过期因为太耗CPU它不用

但是惰性这种又会可能导致大量无效数据堆积在内存里面，我们总得有个办法来解决吧！不能让他一直堆在内存里面啊！

所以我们就有了一个定期过期策略，虽然实时性比不上定时的，但是也足够解决垃圾数据大量堆积在内存的这种情况！

定期过期

先看数据结构　源码 dict.h



redis的hash默认使用的是ht[0]，ht[1]不会初始化和分配空间。

哈希表dictht是用链地址法来解决碰撞问题的。如果节点数量比哈希表的大小要大很多的话，那么哈希表就会退化成多个链表，哈希表本身的性能优势就不再存在,在这种情况下需要扩容。

Redis里面的这种操作叫做rehash。 那么它怎么做rehash的

每过一段时间去执行一次删除过期key，我们来看定期过期到底是怎么实现的！

我们说定期删除，那如果让我们做的话我们应该怎么去做？我们想到的是不是定期去循环找过期的key，然后去删掉！

那么问题又来了

1. 我们去循环谁？是不是所有的key
2. 我们多久循环一次？

好，那么为了让大家更好的理解，我先告诉大家答案，等下如果大家需要看源码，我带大家看下源码！

第一个问题，我们并不是去循环所有的key!

源码（activeExpireCycle expire.c文件下123行）

我先给大家提个问题，我们是过期策略！是要去删除数据库中过期的数据！那么我应该要去拿什么样的数据去进行删除?

对，是不是只要去拿设置了过期时间的数据就行！ server.h 文件下 redisDb 中的expires（660行）！！！

那么问题又来了,我们设置了过期时间的那么多，要一次性全部删除完么?那不得很久很久么

具体实现方式为：

1. 不是一次性把所有设置了过期时间的数据拿出来，而是按hash桶维度取里面取值，取到20个值为止，如果第一个有30个，那么也会取30个！如果一直取不到20，那么最多400个桶
2. 删除取出值的过期key
3. 如果400个桶都取不到值，或者取出的key 删除的比例大于10%,继续上面的操作
4. 每循环16次会去检测时间，超过指定时间就跳出

现在我们第一个问题解决了！那么第二个问题，定期定期，那么多久去做上面那件时间！那么redis里面有个很重要的概念叫做时间事件，那么这个时间事件是什么意思了，就是定时去做一些事情，那么redis里面有个方法叫serverCron()，文件server.c中；就是它的时间事件去调用的

它里面干了很多事情，比如

1. 更新服务器的各类统计信息，比如时间、内存占用、数据库占用情况等
2. 清理数据库中的过期键值对。
3. 关闭和清理连接失效的客户端
4. 尝试进行持久化操作

那么这个时间事件多久去执行一次呢，其实是由你们自己决定的！redis.conf 中通过 hz 配置，hz代表的意思是每秒执行多少次！默认10次，也就是100ms我们就会去执行定期过期！！

定期过期的逻辑，我也给大家简单画了个图

https://www.processon.com/view/link/5fbf736d5653bb1d54fc233e

Redis淘汰策略

淘汰策略，我们刚才讲了。。。是因为我们redis中的数据都没有过期，但是内存有大小，所以我们得去不得已淘汰一些没有过期的数据！！

那么怎么去淘汰了，我们刚才讲了冰箱其实也是相当于一个缓存容器，放菜！！

那么如果现在冰箱里面的菜都是好的没过期的，但是你家冰箱满了，买新冰箱又来不及，要去仍菜或者把它吃掉！就是要清理菜！你们会怎么清理！

对，我们redis里面都有相关实现！！！

官网地址：https://redis.io/topics/lru-cache

我们从官网地址可以看到，最开始有6种，4.0版本后添加了LFU，它们到底是啥，我们来看，几个组成部分，前面有个前缀volatile跟allkeys，这个什么意思了，这个其实一个是所有key跟有过期时间的key！后面的才是真正的淘汰算法，前面代表的是淘汰的范围！那么我们是不是只要弄清楚后面的算法就OK了！

一个LRU，一个LFU，random这个我不讲，因为聪明的你们都知道，就是随机！ttl就是根据过期时间来，即将过期的删除！noeviction 这个就是默认不删除！满没满我不关心，满了就满了，反正我是不去淘汰！其他的都很好理解！

所以等下我会给大家重点讲解下LRU跟LFU的原理好吧！

既然它给出了这么多方案，肯定有个地方去配置我用哪种对不对！好，大家一猜应该就知道，肯定在config文件里面配置不会错！对，就是在那里！

```
# maxmemory-policy noeviction
```

默认就是不淘汰，如果满了，能读不能写！就跟你冰箱满了一样，能吃，不能放！

LRU

Lru，Least Recently Used 那么它的一个衡量标准是啥？时间对不对！

翻译过来是最久未使用，根据时间轴来走，仍很久没用的数据只要最近有用过，我就默认是有效的。

我们先来看一个对象 redisObject ，这个对象就是我们redis的所有数据结构的对外对象！那么它里面有个字段叫做lru！！

源码：server.h 620行

```
typedef struct redisObject {
unsigned type:4;
unsigned encoding:4;
unsigned lru:LRU_BITS; /* LRU time (relative to global
lru_clock) or
\* LFU data (least significant 8 bits frequency
\* and most significant 16 bits access time). */
int refcount;
void *ptr;
} robj;
```

每个对象都有1个LRU的字段，看它的说明，好像LRU跟我们后面讲的LFU都跟这个字段有关！！并且这个lru的时候代表的是一个时间相关的内容！！！

那么我们大概能猜出来，redis去实现lru肯定跟我们这个对象的lru相关！！

那么lru刚刚我们讲了，是根据时间去排序删除的，那么怎么去判断一个redis数据有多久没访问了了！

首先，我告诉大家，这个lru在LRU算法的时候代表的是这个数据的访问时间的秒单位！！但是只有24bit，无符号位，所以这个lru记录的是它访问时候的秒单位时间的后24bit！

相当于

```
        long timeMillis=System.currentTimeMillis();
        System.out.println(timeMillis/1000); //获取当前秒

        System.out.println(timeMillis/1000 & ((1<<24)-1));
    //获取秒的后24位
```

那么，我们知道了这个线索，那么我们要得到这个对象多久没访问了，我们是不是就很简单，用我当前的时间-这个对象的访问时间就可以了！！！但是这个访问时间是秒单位时间的后24bit！所以，也是用当前的时间的秒单位的后24bit去减!

好，这个是我们想的，但是有些问题是我们想不到的！！！！！

我们刚才讲了，是获取当前时间的最后24位，那么当我们最后的24bit都是1了的时候，时间继续往前走，那么我们获取到的时间是不是就是24个0了！

举个例子

```
111111111111111110000000000011111110   假如这个是我当前秒单位的时间，获取后8位   是   11111110
111111111111111110000000000011111111    获取后8位   是   11111111
111111111111111110000000000100000000    获取后8位   是   00000000
```

所以，它有个轮询的概念，它如果超过24位，又会从0开始！所以我们不能直接的用系统时间秒单位的24bit位去减对象的lru,而是要判断一下！！！那么怎么判断了，我等下给大家看图再详解

还有一点是，为了性能，我们系统的时间不是实时获取的，而是用我们redis的那个时间事件来获取，来告诉我是哪个！！！是不是serverCorn，所以，我们这个时间获取100ms去获取一次！！！

为了大家能更好的理解，我这边也画了图

https://www.processon.com/view/link/5fbe0541e401fd2d6ed8fc38

好，现在我们知道了原来redis对象里面原来有个字段是记录它访问时间的，那么接下来肯定有个东西去跟这个时间去比较，拿到差值！

我们去看下源码evict.c文件

```
unsigned long long estimateObjectIdleTime(robj *o) {
    //获取秒单位时间的最后24位
    unsigned long long lruclock = LRU_CLOCK();
    //因为只有24位，所有最大的值为2的24次方-1
    //超过最大值从0开始，所以需要判断lruclock（当前系统时间）跟缓存对象的lru字段的大小
    if (lruclock >= o->lru) {
        //如果lruclock>=robj.lru，返回lruclock-o->lru，再转换单位
        return (lruclock - o->lru) * LRU_CLOCK_RESOLUTION;
    } else {
        //否则采用lruclock + (LRU_CLOCK_MAX - o->lru)，得到对象的值越小，返回的值越大，越大越容易被淘汰
        return (lruclock + (LRU_CLOCK_MAX - o->lru)) *
                    LRU_CLOCK_RESOLUTION;
    }
}
```

我们发现，跟对象的lru比较的时间也是serverCron下面的当前时间的秒单位的后面24位！但是它有个判断，有种情况是系统时间跟对象的LRU的大小，因为最大只有24位，会超过！！

所以，我们可以总结下我们的结论

1.Redis数据对象的LRU用的是server.lruclock这个值，server.lruclock又是每隔100ms生成的系统时间的秒单位的后24位！所以server.lruclock可以理解为延迟了100ms的当前时间秒单位的后24位！

2. 用server.lruclock 与 对象的lru字段进行比较，因为server.lruclock只获取了当前秒单位时间的后24位，所以肯定有个轮询。所以，我们会判断server.lruclock跟对象的lru字段进行比较，如果server.lruclock>obj.lru，那么我们用server.lruclock-obj.lru,否则

server.lruclock+(LRU_CLOCK_MAX-obj.lru),得到lru越小，那么返回的数据越大，相差越大的就会被淘汰！

LFU

LFU，英文 Least Frequently Used，翻译成中文就是最不常用，不常用，它的衡量标准就是次数

但是LFU的有个致命的缺点！就是它只会加不会减！ 所以我们来看redis它是怎么实现的！怎么解决这个缺点！

我们还是来看我们的redisObject(server.h 620行)

```
typedef struct redisObject {
unsigned type:4;
unsigned encoding:4;
unsigned lru:LRU_BITS; /* LRU time (relative to global
lru_clock) or
* LFU data (least significant 8 bits frequency
* and most significant 16 bits access time). */
int refcount;
void *ptr;
} robj;
```

我们看它的lru，它如果被用作LFU的时候！它前面16位代表的是时间，后8位代表的是一个数值，frequency是频率，应该就是代表这个对象的访问次数，我们先给它叫做counter。

那么这个16位的时间跟8位的counter到底有啥用呢？8位我们还能猜测下，可能就是这个对象的访问次数！

那么，如果让我们去猜，可能就比较简单了！！我们淘汰的时候，是不是就是去根据这个对象的次数，按照小的就去给它淘汰掉！！！

其实，差不多就是这么个意思!

还有个问题，如果8位用作访问次数的话，那么8位最大也就2的8次方，就是255次，够么？如果，按照我们的想法，肯定不够，那么redis去怎么解决呢！

既然不够，那么让它不用每次都加就可以了,能不能让它值越大，我们加的越慢就能解决这个问题！！！

对，redis就是这么干的，但是，redis还加了个东西，让你们自己能主宰它加的速率！！这个东西就是lfu-log-factor！它配置的越大，那么对象的访问次数就会加的越慢！！

(evict.c 328行)

```
uint8_t LFULogIncr(uint8_t counter) {
    //如果已经到最大值255，返回255 ，8位的最大值
   if (counter == 255) return 255;
   //得到随机数（0-1）
double r = (double)rand()/RAND_MAX;
//LFU_INIT_VAL表示基数值（在server.h配置）
double baseval = counter - LFU_INIT_VAL;
//如果达不到基数值，表示快不行了，baseval =0
if (baseval < 0) baseval = 0;
//如果快不行了，肯定给他加counter
//不然，按照几率是否加counter，同时跟baseval与lfu_log_factor相关
//都是在分子，所以2个值越大，加counter几率越小
    double p = 1.0/(baseval*server.lfu_log_factor+1);
    if (r < p) counter++;
    return counter;
}
```

所以，LFU加的逻辑我们可以总结下：

1. 如果已经是最大值，我们不再加！因为到达255的几率不是很高！可以支撑很大很大的数据量！
2. counter属于随机添加，添加的几率根据已有的counter值和配置lfu-log-factor相关，counter值越大，添加的几率越小，lfu-log-factor配置的值越大，添加的几率越小！

还有,这个前16bit时间到底是干嘛的！！我们现在还不知道，我们刚才讲了。传统的LFU只能加，不会减

那么我们想下，这个时间是不是就是用来减次数的！！！

大家有玩王者充钱的么，如果充钱的同学应该知道，如果你很久很久不充钱的话，你的vip等级会降！！！诶，这个是不是就能解决我们的次数只能加不能减的问题！并且这个减还是根据你多久时间没充钱来决定的！！！

所以，我们可以大胆猜下，这个前16bit的时间是不是也记录了这个对象的时间，然后根据这个时间判断这个对象多久没访问了就去减次数了！！！

没错，你猜的都是对的！我们的这个16bit记录的是这个对象的访问时间的分单位的后16位，每次访问或者操作的时候，都会去跟当前时间的分单位的后16位去比较，得到多少分钟没有访问了！然后去减去对应的次数！

那么这个次数每分钟没访问减多少了，就是根据我们的配置lfu-decay-time

这样就能够实现我们的LFU，并且还解决了LFU不能减的问题！！

这个我给大家也画了个图

https://www.processon.com/view/link/5fbf46efe401fd2d6edba736

那么我们可以去看下减的源码

```
unsigned long LFUDecrAndReturn(robj *o) {
  //lru字段右移8位，得到前面16位的时间
unsigned long ldt = o->lru >> 8;
//lru字段与255进行&运算（255代表8位的最大值），
//得到8位counter值
unsigned long counter = o->lru & 255;
//如果配置了lfu_decay_time，用LFUTimeElapsed(ldt) 除以配置的值
//LFUTimeElapsed(ldt)源码见下
//总的没访问的分钟时间/配置值，得到每分钟没访问衰减多少
unsigned long num_periods = server.lfu_decay_time ?
LFUTimeElapsed(ldt) / server.lfu_decay_time : 0;
if (num_periods)
//不能减少为负数，非负数用couter值减去衰减值
  counter = (num_periods > counter) ? 0 : counter -
num_periods;
  return counter;
}
```

LFUTimeElapsed方法源码(evict.c)

```
//对象ldt时间的值越小，则说明时间过得越久
unsigned long LFUTimeElapsed(unsigned long ldt) {
//得到当前时间分钟数的后面16位
unsigned long now = LFUGetTimeInMinutes();
//如果当前时间分钟数的后面16位大于缓存对象的16位
//得到2个的差值
if (now >= ldt) return now-ldt;
//如果缓存对象的时间值大于当前时间后16位值，则用65535-ldt+now得到差值
    return 65535-ldt+now;
}
```

所以，LFU减逻辑我们可以总结下：

1. 我们可以根据对象的LRU字段的前16位得到对象的访问时间（分钟），根据跟系统时间比较获取到多久没有访问过！
2. 根据lfu-decay-time（配置），代表每分钟没访问减少多少counter,不能减成负数

持久化机制

RDB持久化

RDB 是 Redis 默认的持久化方案。RDB快照（Redis DataBase），当满足一定条件的时候，会把当前内存中的数据写入磁盘，生成一个快照文件dump.rdb。

我们先看下RDB持久化相关配置

```
# 文件路径，
dir ./
# 文件名称
dbfilename dump.rdb
# 是否是 LZF 压缩 rdb 文件
rdbcompression yes
# 开启数据校验 sht
rdbchecksum yes
```

Redis重启会通过dump.rdb文件恢复数据。那那个一定的条件是啥呢？到底什么时候写入rdb 文件？

1. 自动触发
    1. 配置规则触发

       redis.conf， SNAPSHOTTING配置，其中定义了触发把数据保存到磁盘的 触发频率。 如果不需要 RDB 方案，注释 save 或者配置成空字符串""。

       ```
       save 900 1 # 900 秒内至少有一个 key 被修改（包括添加）
       save 300 10 # 300 秒内至少有 10 个 key 被修改
       save 60 10000 # 60 秒内至少有 10000 个 key 被修改
       ```

       注意上面的配置是不冲突的，只要满足任意一个都会触发。
    2. shutdown触发,保证服务器正常关闭，关闭的时候不会造成数据丢失。
    3. flushall RDB文件是空的，没什么意义。
2. 手动触发

   如果我们需要重启服务或者迁移数据，这个时候就需要手动触RDB快照保存。所以redis也提供了2种手动保存RDB快照的指令。
    1. Save

       save在生成快照的时候会阻塞当前Redis服务器，Redis不能处理其他命令。如果内存中的数据比较多，会造成Redis长时间的阻塞。生产环境不建议使用这个命令。为了解决这个问题，Redis 提供了第二种方式。
    2. bgsave

       执行bgsave时，Redis会在后台异步进行快照操作，快照同时还可以响应客户端请求。

       具体操作是Redis进程执行fork（创建进程函数）操作创建子进程(copy-on-write)，RDB持久化过程由子进程负责，完成后自动结束。它不会记录 fork
       之后后续的命令。阻塞只发生在fork阶段，一般时间很短。用 lastsave 命令可以查看最近一次成功生成快照的时间。

我们接下来演示一个RDB快照恢复数据的一个过程！（我的快照在src下面）

我们首先添加一份数据并且备份到RDB

```
redis> set k1 1
redis> set k2 2
redis> set k3 3
```

查看数据是否存在

```
redis>keys *
```

我们进行shutdown操作触发RDB快照

```
redis> shutdown
```

对现有RDB数据进行备份cp

```
redis>cp dump.rdb dump.rdb.bak
```

启动redis

```
redis> src/redis-server& redis.conf
```

-p 6381

发现数据都还在，现在模拟数据丢失

```
redis> flushall
```

停服务器再启动

```
redis> shutdown

redis> src/redis-server& redis.conf
```

发现数据已经丢失，我们现在就要从我们备份的数据恢复，先关闭

```
redis> shutdown
```

删除原RDB备份数据cd .

```
redis>rm dump.rdb
```

将备份数据改名为dump.rdb

```
mv dump.rdb.bak dump.rdb
```

重启服务

```
src/redis-server& redis.conf
```

我们知道了RDB的实现的原理逻辑，那么我们就来分析下RDB到底有什么优劣势。

一、优势

1．RDB是一个非常紧凑(compact类型)的文件，它保存了redis在某个时间点上的数据集。这种文件非常适合用于进行备份和灾难恢复。

2．生成RDB文件的时候，redis主进程会fork()复刻一个子进程来处理所有保存工作，主进程不需要进行任何磁盘IO操作。

3．RDB在恢复大数据集时的速度比AOF的恢复速度要快。

二、劣势

1．RDB方式数据没办法做到实时持久化/秒级持久化。因为bgsave每次运行都要执行 fork 操作创建子进程，频繁执行成本过高。

2．在一定间隔时间做一次备份，所以如果redis意外down掉的话，就会丢失最后一次快照之后的所有修改（数据有丢失）。

所以，针对与这种情况，redis又提供了一种持久化机制，就是AOF持久化机制！

AOF持久化

Append Only File的简称，只加载文件，这个方案在redis默认是不开启的。

AOF是怎么恢复的？AOF采用日志的形式来记录每个写操作，并追加到文件中。开启后，执行更改 Redis数据的命令时，就会把命令写入到AOF文件中。Redis重启时会根据日志文件的内容把写指令从前到后执行一次以完成数据的恢复工作。

那么它从哪里开启，又有些什么特性！

配置文件redis.conf

```
# 开关
appendonly no

# 文件名
appendfilename "appendonly.aof"
```

同步机制

AOF,会记录每个写的操作，那么问题来了？我难道每次操作命令又得跟磁盘交互呢？不跟mysql一样每次需要跟磁盘交互呢？

当然不行，所以，redis支持几种策略，由你们自己来决定要不要每次都跟磁盘交互

```
# appendfsync always   表示每次写入都执行fsync(刷新)函数
appendfsync everysec   每秒执行一次fsync函数
# appendfsync no    由操作系统保证数据同步到磁盘，速度最快
```

默认1s一次，最多有1s丢失

重写机制（rewrite）

为什么要重写?

比如，我们用分布式锁，指令是setnx,然后设置过期日期；由于是每次都是命令追加。那么1年后，10年后，我们会发现这个aof文件里面都是这样的指令！！

我一个1T的aof文件，一直在执行这个2个指令！！所以我们需要重写。

AOF文件重写并不是对原文件进行重新整理，而是直接读取服务器现有的键值对，然后用一条命令去代替之前记录这个键值对的多条命令，生成一个新的文件后去替换原来的AOF文件。

怎么重写

4.0版本之前，比较指令，去除无效指令

假如：aof这样几个指令

```
redis> lpush huihuilist a

redis> lpush huihuilist b c d

redis> lpop huihuilist
```

就会重写成

```
redis> lpush huihuilist a b c
```

我们发现，这种方法效率很低，需要一个一个区比对！

所以，4.0之后引入了RDB 跟AOF 混合的模式，让RDB跟AOF一起使用，因为RDB的速度快，那么我们就引入这个特点，根据RDB模式，将以前的指令以二进制的方式覆盖到aof,后面写入的继续追求到文件后面。（混合模式默认是开启，也可以关闭）

```
aof-use-rdb-preamble yes   //是否开启RDB与AOF混合模式
```

什么时候重写

配置文件redis.conf

```
# 重写触发机制
auto-aof-rewrite-percentage 100

auto-aof-rewrite-min-size 64mb    最开始aof文件必须要达到这个文件时才触发，后面的每次重写就不会根据这个变量了
```

举例说明上述配置的意义

咕泡出品

在 aof 文件小于64mb的时候不进行重写，当到达64mb的时候，就重写一次。重写后的 aof 文件可能是20mb。上面配置了auto-aof-rewrite-percentag为100，即 aof 文件到了40mb的时候，又开始重写一次。以此类推。

我们知道了AOF的实现原理，我们来分析下它的优缺点。

一、优点

能最大限度的保证数据安全，就算用默认的配置everysec，也最多只会造成1s的数据丢失。

二、缺点

数据量比RDB要大很多，所以性能没有RDB好，没有一个性能保证！

那我们平时开发中应该使用哪种持久化呢？

如果可以忍受一小段时间内数据的丢失，毫无疑问使用 RDB 是最好的，定时生成RDB快照（snapshot）非常便于进行数据库备份， 并且 RDB 恢复数据集的速度也要比AOF恢复的速度要快。